

Didžiųjų duomenų apdorojimas Laboratorinis 2

Užduotis:

Nagrinėsime [duomenis](#), aprašančius siuntų pristatymo bei logistikos įmonės veiklą. Duotajame faile pateikti dalinai struktūrizuoti duomenys apie siuntų išvežiotųjų sustojimus, kurie gali būti apibūdinami šias parametrais:

"marsrutas", "sustojimo data", "sandelio id", "Firma", "Marsruto tipas", "Masinos tipas", "sustojimo tipas", "sustojimo savaitės diena", "laikas", "Sustojimo numeris", "siuntu skaicius", "svoris", "svorio grupe", "geografine zona", "pasto kodas", "Aptarnavimo grupe", "tipas", "Laikas iki sustojimo", "Laikas po sustojimo", "Uzkrovimo tipas", "Ar reikalingos paletes", "Laukia", "Sustojimo klientu skaicius", "sustojimo klientu sarasas", "kaina procentas", "kaina vienetais".

Naudodami *Apache Spark RDD API* atlikite šias užduotis:

- Slenkstinis lygmuo (5-6): Suskaičiuokite mažiausią, didžiausią ir vidutinį (aritmetinis vidurkis) siuntų svorį (laukas "svoris") skirtingose svorio grupėse (laukas "svorio grupe").
- Tipinis lygmuo (7-8): Raskite maršrutus, kurie aplanko daugiau nei vieną geografinę zoną (laukas "geografine zona"). Koks tai procentas nuo visų maršrutų? Raskite atvejus, kai tai daroma ta pačia diena (laukas "sustojimo data"). Koks tai procentas?
- Puikus lygmuo (9-10): Sudarykite lentelę, kurioje matytųsi kiek pristatyta siuntų ("siuntu skaicius") bei aptarnauta klientų ("Sustojimo klientu skaicius") skirtinguose geografinėse zonose ("geografine zona") skirtingomis savaitės dienomis ("sustojimo savaitės diena"). Palyginkite užduoties sprendimo laiką su MapReduce versiją.

Ataskaita

Užduočiai atlikti, pirmiausia reikia išnagrinėti duomenis. Čia pateiktas vieno įrašo sutrumpintas pavyzdys:

```
{{pasto kodas=11208}{Sustojimo klientu skaicius=1}{sustojimo savaitės diena=2}  
{laikas=12:09:07}{Sustojimo numeris=5}{geografine zona=Z1}{sustojimo data=2018-01-02}  
{svoris=41}{Laikas po sustojimo=27.48}{siuntu skaicius=1}{kaina vienetais=2.854}}{...
```

Kiekvienas įrašas atskirtas riestais skliausteliais "{ }", kaip ir kiekvienas įrašo parametras. Tam, kad atskirti įrašus ir kiekvieną parametą, parašyta pagalbinė funkcija, kuri priima įrašų eilutę, ir sąrašą parametrų pavadinimų, kuriuos atskirti.

```

import re

class Helpers:
    @staticmethod
    def extract_entries(line: str, params_to_extract=None):
        if not params_to_extract:
            raise ValueError("Error: Specify parameter names to extract.")

        entries = re.findall(r'\{\{.*?\}\}', line)

        for entry in entries:
            # Extract key=value pairs from the entry
            pairs = re.findall(r'(\w+(?:\s\w+)*)(=[^,]+)?', entry)
            entry_dict = {key.strip(): (value.strip() if value else "") for
key, value in pairs}

            # Skip entry if any required param is missing or has an empty
value
            if all(var in entry_dict and entry_dict[var].strip() for var in
params_to_extract):
                filtered_dict = {}
                for var in params_to_extract:
                    value = entry_dict[var]
                    try:
                        value = float(value)
                    except ValueError:
                        pass
                    filtered_dict[var] = value

                yield filtered_dict

```

Išankstinės nuostatos:

Duotame įrašų rinkinyje, ne kiekvienas parametras užpildytas, todėl, jei, pagal duotus raktus, trūksta bent 1 parametro ar jo vertės, visas įrašas praleidžiamas.

1 Uždutis. Suskaičiuoti mažiausią, didžiausią ir vidutinį siuntų svorį skirtingose svorio grupėse.

Reikalingi parametrai (svorio grupė, svoris).

Užduočiai atlikti, panaudoti RDD Api:

- map - išskirti svorio grupę kaip raktą, svorį kaip vertę.
- groupByKey - sugrupuoti visus svorius pagal svorio grupę.
- mapValues - atlikti skaičiavimus su svorių sąrašais.

```
lines = sc.textFile("./data/duom_full.txt")
entries = lines.flatMap(lambda line: list(Helpers.extract_entries(line,
[svoris, svorio_grupe])))

group_weight_pairs = entries.map(lambda entry: (entry[svorio_grupe],
entry[svoris]))

grouped = group_weight_pairs.groupByKey()

def calculate_stats(weights):
    weights_list = list(weights)
    return (
        min(weights_list),
        max(weights_list),
        sum(weights_list) / len(weights_list)
    )

result = grouped.mapValues(calculate_stats)

formatted_results = result.map(lambda x: {
    "weight_group": x[0],
    "min_weight": x[1][0],
    "max_weight": x[1][1],
    "avg_weight": x[1][2]
})

results = formatted_results.collect()

# Convert to DataFrame for visualization
result_df = spark.createDataFrame(formatted_results)
result_df.show()
```

Rezultatas:

avg_weight	max_weight	min_weight	weight_group
5.964402623894628	50.0	0.0	<50
759.2116686954745	6896.65	300.05	>300
110.03327885846451	300.0	50.05	<300

2 Uždutis. Rasti maršrutus, kurie aplanko daugiau nei vieną geografinę zoną ir kokį procentą jie užima?

Rasti atvejus, kai maršrutai aplanko daugiau nei vieną geografinę zoną, tą pačią dieną ir kokį procentą tokie maršrutai užima?

Reikalingi parametrai: maršrutas, geografinė zona, sustojimo data.

Panaudoti RDD Api:

- distinct - surasti unikalius maršrutus.
- count - surasti unikalių maršrutų kieki.
- map - nustatyti maršrutą kaip raktą, tuple duomenų objektą iš geografinės zonos, sustojimo datos, kaip vertę.
- groupByKey - sugrupuoti geografines zonas ir sustojimo datas pagal maršrutus.
- filter - atrinkti maršrutus, kurie aplanko daugiau nei vieną geografinę zoną.

```
# Rasti unikalius maršrutus
all_routes = entries.map(lambda entry: entry[marsrutas]).distinct()
total_routes = all_routes.count()
if total_routes == 0:
    assert("Rasta 0 maršrutų.")

route_zone_date_pairs = entries.map(lambda entry: (
    entry[marsrutas],
    (entry[geografine_zona], entry[sustojimo_data])
))

# Grupuojuame pagal maršrutą ir randame unikalias zonas
def analyze_zones_and_dates(zone_date_pairs):
    unique_zones = set(zone for zone, _ in zone_date_pairs)

    zones_by_date = {}
    for zone, date in zone_date_pairs:
        if date not in zones_by_date:
            zones_by_date[date] = set()
        zones_by_date[date].add(zone)
```

```

    # Rasti dienas, kuriomis aplankyta daugiau nei viena zona
    multi_zone_dates = [date for date, zones in zones_by_date.items() if
len(zones) > 1]

    return {
        "unique_zones": unique_zones,
        "zones_count": len(unique_zones),
        "has_multi_zones": len(unique_zones) > 1,
        "zones_by_date": zones_by_date,
        "multi_zone_dates": multi_zone_dates,
        "has_multi_zones_same_day": len(multi_zone_dates) > 0
    }

route_analysis = route_zone_date_pairs
    .groupByKey()
    .mapValues(analyze_zones_and_dates)

# Rasti maršrutus, kurie aplanko daugiau nei 1 zoną
multi_zone_routes = route_analysis.filter(lambda x: x[1]["has_multi_zones"])
multi_zone_count = multi_zone_routes.count()

# Rasti maršrutus, kurie aplanko daugiau nei 1 zoną tą pačią dieną
multi_zone_same_day_routes = route_analysis.filter(lambda x:
x[1]["has_multi_zones_same_day"])
multi_zone_same_day_count = multi_zone_same_day_routes.count()

# Rasti procentus
multi_zone_percent = (multi_zone_count / total_routes) * 100
multi_zone_same_day_percent = (multi_zone_same_day_count / total_routes) *
100

```

Rezultatas:

Iš viso unikalių maršrutų: 424

Maršrutai, aplankantys daugiau nei vieną geografinę zoną: Skaičius: 395

Procentai: (93.16%)

Maršrutai, aplankantys daugiau nei vieną geografinę zoną tą pačią

dieną: Skaičius: 394 Procentai: (92.92%)

3 Uždutis. Sudaryti lentelę, kurioje matytųsi kiek pristatyta siuntų bei aptarnauta klientų skirtinguose geografinėse zonose, skirtingomis savaitės dienomis. Palyginkite užduoties sprendimo laiką su MapReduce versija.

Reikalingi parametrai: geografinė zona, sustojimo savaitės diena, siuntų skaičius, sustojimo klientų skaičius.

Panaudoti RDD Api:

- map - geografinės zonos ir sustojimo savaitės dienos tuple priskirtas kaip raktas, siuntų skaičius ir sustojimo klientų skaičius - reikšmė.
- reduceByKey - apskaičiuoti siuntų ir sustojimo klientų sumą.

```
from collections import defaultdict
lines = sc.textFile("./data/duom_full.txt")

entries = lines.flatMap(lambda line: list(Helpers.extract_entries(line, [
    geografine_zona,
    sustojimo_savaites_diena,
    siuntu_skaicius,
    sustojimo_klientu_skaicius
])))

# Sukuriame rakto-reikšmės poras: (zona, diena) -> (siuntų skaičius, klientų
skaičius)
zone_day_pairs = entries.map(lambda entry: (
    (entry[geografine_zona], entry[sustojimo_savaites_diena]),
    (entry[siuntu_skaicius], entry[sustojimo_klientu_skaicius])
))

zone_day_sums = zone_day_pairs.reduceByKey(lambda a,b: (a[0]+b[0],
a[1]+b[1]))

results = zone_day_sums.map(lambda x: {
    "geografine_zona": x[0][0],
    "savaites_diena": int(x[0][1]),
    "pristatyta_siuntiniu": int(x[1][0]),
    "aptarnauta_klientu": int(x[1][1])
})

result_df = spark.createDataFrame(results)
result_df = result_df.orderBy("geografine_zona", "savaites_diena")

result_df.show()
```

Rezultatas:

aptarnauta_klientu	geografine_zona	pristatyta_siuntiniu	savaite_diena
41580	Z1	86464	1
60167	Z1	123938	2
62241	Z1	126872	3
48430	Z1	98156	4
45999	Z1	88663	5
797	Z1	1754	6
11751	Z2	19525	1
18499	Z2	31377	2
18545	Z2	30715	3
14190	Z2	23727	4
14027	Z2	23883	5
72	Z2	136	6
11002	Z3	17149	1
17184	Z3	25642	2
17246	Z3	25785	3
12887	Z3	19499	4
12304	Z3	18423	5
23	Z3	26	6

MapReduce rezultatas:

```
geografine zonaZ1=1_sustojimo savaites diena=1_Sustojimo klientu skaicius      41580
geografine zonaZ1=1_sustojimo savaites diena=1_siuntu skaicius      86464
geografine zonaZ1=1_sustojimo savaites diena=2_Sustojimo klientu skaicius      60167
geografine zonaZ1=1_sustojimo savaites diena=2_siuntu skaicius      123938
geografine zonaZ1=1_sustojimo savaites diena=3_Sustojimo klientu skaicius      62241
geografine zonaZ1=1_sustojimo savaites diena=3_siuntu skaicius      126872
geografine zonaZ1=1_sustojimo savaites diena=4_Sustojimo klientu skaicius      48430
geografine zonaZ1=1_sustojimo savaites diena=4_siuntu skaicius      98156
geografine zonaZ1=1_sustojimo savaites diena=5_Sustojimo klientu skaicius      45999
geografine zonaZ1=1_sustojimo savaites diena=5_siuntu skaicius      88663
...
```

Spark RDD metodu, rezultatas gautas per 14,5s MapReduce užtruko 28,6s, todėl šioje užduotyje Spark implementacija buvo beveik **2 kartus** greitesnė. Projekto nuoroda:

<https://github.com/EmilisGit/Apache-pyspark/tree/jupyter>